Method Parameters

call by value

call by reference

Call by just value → call by location ~~refers~~ present here.

Java → call by value

Primitives , Objects

Primitives → values are passed

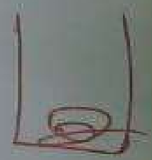
Objects → values are references → So they can look, alter state But see it

Primitives

int x = 8;

triple x(x);

x = 3 * x



Class Design

Always keep Data private

Always initialize data

Don't use too many basic types of class

Not all fields needs access & mutators

Break up class → Too many responsibility

Make name of your class & methods
reflect responsibilities

Prefer immutables

Field comm

public → static comment

General comment

@author → author entry
@version → version entry
@ par → par

package comment

package directory

package-info.java

Comment extraction

HTML files

extract to library

Executable jar file

e option → jar command → entry point of program specified

Documentation comments

Javadoc → HTML documentation

Comment Insertion

Javadoc

Class comments

Method comments

@param variable description

@return return section of comment

@throws Exception

Setting class path

-classpath or -cp

--class-path

JAR Files

□ Java application

single file

root a directory structure of classfiles

Creating JAR File

jar cvf jarfilename file....

Manifest

images
other
resources

manifest
special features of archive

directory

class file

base directory

current directory

Jar file
inside direct

set
class path

You can import one public class
from other packages

Source file

one public class

names of file & public class
match

Import non-public class
in current package

Compiler search all source files

If you don't provide private
access modifier

that's why
Variable
→ private

Classes are

packa ———
class
access it

mu

packyaccess

Classpaths

Subdirectories of file system

Class files

Java Archive file — Simulate any files subdirectory
Compressed
format

Compiler
↳ don't care directory structure

doesnot depen on package → will compile

Virtual machine
won't find classes
package don't match directories

Package Access

public → access any
private → access only the class

If you don't provide a access modifier
feature can be access all methods
in same package.

Variables → private

If you don't provide private

default → package access

Addition of class
to package

package com.______;
public class Sum {

}

If you don't put package statement
in source file
↓
{unnamed package}

place source files → subdirectory

Full package name

Compiler
↓
files file separ

Java Interpreter
↓
Class
dot separators

The only time worry import

when both packg same class

~~[illegible]~~ import appropriate one seperately

Locating classes → package

↓

activities → compiler

bytecodes → full name of package

refer to other class

Static imports

Form of imports both static

fields methods not just class

import static java.lang.Math.*;

$\boxed{PI}$

There is no relationship
b/w nested package

How to import class

class can use all classes
from own package -> public class
from other pa

fully qualified name
package name + classname

java
util
java.util.Scanner;
scanner
class

import -> use a specific class
or whole package

at top of source files
below any package
statements

Packages

Group of classes collection

packages

Yours own

Libraries provided by others

separates.

Package Names

main reason → class to have a unique name

Class A

Class A

Both name should be same But different type

Package Impl1:
Class A

Pack Impl2
Class A

Impl1.ClassA
Impl2.ClassA

full name

com.google [illegible]